

HYDRA: High-Yield Data Reorganization & Acceleration

Mid-Term Report

Nafil Atiq*, Ruohan Dong*, Matthew Nutt*, Giovanni Sirtori*, Ithzel Toscano*, Yulia Zhou*

*Department of Electrical & Computer Engineering

Rice University, Houston, Texas

{na115, rd100, mc5, gs86, ijt2, yz305}@rice.edu

Abstract—This document covers the midterm work completed by the RISC-V Master’s Capstone team on the HYDRA project. Results from DFT and verification frameworks are available, but work in the areas of DMA design and RTL review only involve planning considerations. Future work on the DFT and verification flows will involve expanding the capabilities of their respective systems, and future work on the RTL side will involve the integration of a preliminary DMA system into a third-party RISC-V core.

Index Terms—RISC-V, DMA, RTL, UVM, Verilog, SystemVerilog

I. INTRODUCTION

Modern AI inference systems are increasingly bottlenecked by irregular memory access patterns in operations like embedding lookup, sparse attention, and tensor layout conversion. Traditional DMA engines improve bandwidth but lack semantic awareness, while full accelerators are costly and inflexible. We propose the High-Yield Data & Acceleration (HYDRA), a programmable DMA engine with an AI-aware datapath that performs in-flight transformations. HYDRA implements three hardware-accelerated modes: scatter-gather for irregular access prefetch, corner-turning for zero-overhead matrix transpose, and real-time quantization to reduce memory traffic. HYDRA enables CPU orchestration while offloading memory-intensive operations to dedicated hardware. We expect to achieve 5-10× reduction in execution cycles and DRAM traffic compared to CPU-only solutions, demonstrating that datapath-centric design can address AI inference bottlenecks more efficiently than compute-heavy accelerators.

II. METHODS

A. RTL Review

This team has inherited work from previous semesters of the capstone class, including a repository of RTL code for a RISC-V core. The design implemented the basic RV32I ISA and featured several additions such as branch prediction and a cache system. A thorough review of this repository would be used to determine if it would be wiser to continue developing on top of the inherited code, or to build our DMA system on a third-party design that would be more refined. The repository in its entirety is available online.¹

B. Design for Testability (DFT)

The processor design incorporates a scan-based Design for Testability architecture to improve the controllability and observability of internal pipeline states during testing. At the RTL level, the major pipeline registers were converted into scan registers and connected to form a single scan chain. Specifically, four pipeline registers were connected sequentially. Additional control signals were introduced to enable scan shifting and state observation.

A simulation testbench was developed to verify the scan functionality. When `scan_en` is asserted, the pipeline registers operate as a shift register, allowing internal states to be serially shifted through the scan chain. In normal operation, the processor captures functional data into the registers. QuestaSim simulations confirmed correct scan shifting and state propagation.

To evaluate testability, a lightweight scan-based fault testing framework was implemented following the standard scan testing sequence: reset, scan-in, capture, and scan-out. During the scan-in phase, a test pattern is shifted into the scan chain. The design then executes one functional clock cycle (capture phase), and the resulting register states are shifted out for analysis.

For each selected scan bit, two simulations were executed: a golden run without faults and a faulty run where the selected bit was forced to the fault value. A fault was considered detected if the scan-out results differed between the two runs.

To reduce simulation runtime, a sampling-based fault evaluation strategy was used. Rather than testing all 1604 scan bits, representative bits were selected from each pipeline register. This approach provides a preliminary estimation of scan-based fault detectability across the pipeline stages.

C. Cache Verification Framework (DCache)

The project focuses on the functional verification of a RISC-V data cache using the SystemVerilog Universal Verification Methodology (UVM). The verification environment evaluates the correctness of cache read/write operations and the interaction between the cache and the memory model. At the RTL level, the cache module is implemented in Verilog and connected to a CPU interface and a simplified memory model. A UVM-based verification environment was developed, including sequence items, a sequencer, a driver, a monitor, and

¹<https://github.com/Rice-MECE-Capstone-Projects/SwitchMCU>

a scoreboard. The sequence items represent CPU memory transactions such as address, data, and access type. The driver converts these transactions into interface-level signals that interact with the cache RTL. During simulation, the monitor observes cache responses and records cache access behavior. All simulations are executed using the QuestaSim simulator, allowing waveform analysis of cache transactions and signal interactions.

D. Core Verification Framework

Initial verification of the processor core used a constrained-random testing framework based on the RISC-V-DV platform. Randomized instruction programs were executed on the DUT, while a scoreboard compared the resulting architectural state against behavioral SystemVerilog reference models to detect functional mismatches. While this approach exercised many instruction sequences, it did not provide a clear metric for how thoroughly processor behavior was explored.

To address this, coverage-driven verification was introduced using SystemVerilog functional covergroups. Covergroups track activity in key components of the core, including the instruction decoder, ALU, and register file. Decoder coverage records observed instruction types, ALU coverage tracks arithmetic and logical operations, and register file coverage monitors reads and writes across the general-purpose registers. Additional coverage points track branch outcomes to verify both taken and not-taken cases.

Coverage statistics are automatically collected during simulation and summarized in coverage reports, allowing coverage percentages to be evaluated and gaps in instruction or datapath behavior to be identified. To validate the coverage framework, the ALU and instruction decoder were first tested in isolation using randomized input stimuli before applying the methodology to the full core.

E. Datapath Architecture

The energy cost of AI inference is dominated by data movement through the memory hierarchy rather than by arithmetic operations [1]. HYDRA addresses this by implementing two hardware acceleration modes: Mode B (Matrix Transposition) and Mode C (32-bit to 8-bit Quantization). Mode B aims to use the block corner-turning technique of Ma et al., partitioning the source matrix into 8×16 basic blocks of 32-bit words, writing each block row-major into an on-chip buffer and reading it column-major to the destination [2]. Two buffers operate in a ping-pong scheme to overlap fetch and transform phases. Mode C implements a combinational rounding-and-clipping unit that packs four 8-bit outputs per 32-bit bus word, quadrupling effective bandwidth per Equation 1. Both modes target 100 MHz, matching the AHB-Lite bus clock domain.

F. SoC Integration Architecture

HYDRA integrates into the Wally RISC-V SoC [3] as a second AHB bus master alongside the CPU, which already

includes a verified L1 cache and AHB interface. The integration work therefore focuses on three additions: a bus arbiter to handle simultaneous requests from the CPU and HYDRA, a non-cacheable scratchpad SRAM that serves as HYDRA's output destination, and the HYDRA MMR slave registered in Wally's existing address map outside the cached memory window. Coherency is resolved by construction: the memory map in Table I enforces non-overlapping cached and non-cached regions in hardware, so HYDRA never writes to a CPU-visible cached address. The overall decomposition follows the front-end / mid-end / back-end model of Benz et al. [4], mapping to HYDRA's MMR register file, address generation unit, and AHB master state machine respectively.

TABLE I
HYDRA SoC MEMORY MAP

Region	Base Address	Size	Attribute
Boot ROM	0x0000_1000	4 KB	Read-only
CLINT	0x0200_0000	64 KB	MMIO
PLIC	0x0C00_0000	64 MB	MMIO
UART / GPIO	0x1000_0000	–	MMIO
HYDRA MMR	0x2000_0000	4 KB	Non-cacheable
Scratchpad SRAM	0x2001_0000	4 MB	Non-cacheable
Main RAM	0x8000_0000	128 MB	Cached (I\$/D\$)

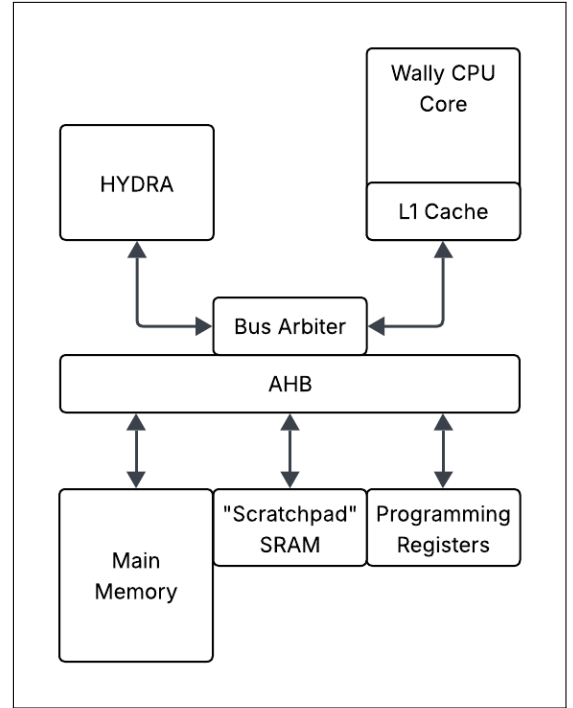


Fig. 1. Full system block diagram.

III. RESULTS

A. RTL Review

After reviewing the existing RTL design, the team found disorganized, poorly documented code, with many incomplete features that were never implemented. Although the core is currently functional (enough for verification and DFT work),

$$B_{\text{eff}} = B_{\text{raw}} \times \frac{W_{\text{src}}}{W_{\text{dst}}} \quad (1)$$

further work with this repository would only result in increased development time due to low readability and technical debt. Along with that, nobody in the current team has prior experience to working on the RTL design of this repository, further complicating efforts to try to understand the previous code. The determination was made to seek out a third-party, open-source RTL design that would facilitate easy integration with our DMA system. Going with a third-party alternative would allow for us to free up time spent trying to understand the code of the current repository, integrating the cache into the core system, and verifying the whole system. This freed time could then be dedicated to working on the RTL design for our proposed DMA system.

B. Design for Testability (DFT)

To evaluate the effectiveness of the implemented scan architecture, a scan-based fault testing framework was executed using simulation. The experiments were performed using the scan testing flow consisting of reset, scan-in, capture, and scan-out phases.

First, the correctness of the scan chain was verified by shifting test patterns through the scan registers. As shown in the scan test output, after 1604 shift cycles, the value injected into the scan chain appears at the scan output, confirming that the entire scan chain of length 1604 bits is functioning correctly.

```
(base) zhuyue@YuliaMacBook-Air-276 F25_core % vvp scan_test.out
T=25000 scan_in=0 scan_out=0
T=35000 scan_in=1 scan_out=0
T=45000 scan_in=0 scan_out=0
T=55000 scan_in=1 scan_out=0
```

Fig. 2. Scan shift verification.

```
After 1604 shifts, scan_out = 0 at time 16060000
T=16065000 scan_in=0 scan_out=0
Extra shift 0: scan_out=1
T=16075000 scan_in=0 scan_out=1
Extra shift 1: scan_out=0
```

Fig. 3. Scan-in and scan-out behavior.

The simulation logs show the sequential propagation of scan values during shifting. The timestamps printed in the output demonstrate that the scan registers correctly operate as a shift register when scan_en is enabled, allowing internal pipeline states to be fully controllable and observable through the scan interface.

After verifying scan functionality, the scan-based fault testing framework was executed. Representative scan bits were sampled from the full scan chain to emulate stuck-at faults (SA0 and SA1). Each selected scan bit was tested twice, once for stuck-at-0 and once for stuck-at-1, by comparing the scan-out results of a fault-free simulation and a faulty simulation.

As shown in the results, some injected faults caused observable differences in the scan-out responses, which were classified as detected faults, while others did not affect the final scan-out state and were classified as missed faults.

```
----START
#
# Cycle_count 0
# pc_o 00000000
# PipeReg0 PC: 00000000 I:(00000013) inst_UNKNOWN rd_1_a 0 rs1_a 0 rs2
_a 0 rs1_d 00000000 rs2_d 00000000 result (0000) imm_1 00000000 branch_en
0 reg_write 0 operand_a 0 alu_res2 0 rd_data 0 d
ata_mem 0 MISSED bit1591 SA0
# [PipeReg3] done. detected=113/216
#
=====
# == ATPG Scan Lite Results ==
# Total faults tested : 216
# Faults detected : 113
# Faults missed : 103
# Fault coverage : 52.31%
# Note: Sampled 108 bits out of 1604 total
#
# ** Note: $finish : atpg_scan_lite.v(214)
# Time: 13871536 ns Iteration: 0 Instance: /atpg_scan_lite
# End time: 14:27:57 on Mar 11, 2026, Elapsed time: 0:01:26
# Errors: 0, Warnings: 17
```

Fig. 4. Scan-based fault testing results.

C. Initial Verification Results (DCache)

Initial simulations were conducted to verify the basic functionality of the verification environment. Test sequences were generated to emulate CPU read and write requests to the cache. The simulation results show that transaction-level requests are correctly translated into cache interface signals. Waveform analysis confirms proper communication between the CPU interface, the cache module, and the memory model. These preliminary results demonstrate that the verification environment is capable of driving cache transactions and observing cache behavior during simulation.

D. Initial Verification Results (Core)

For the decoder, randomized instruction words were applied as inputs and a covergroup tracked the resulting instruction classes based on the decoded control signals. Coverage bins were defined for several instruction categories including arithmetic, logical, and branch, operations. Simulation results showed that multiple bins were triggered, confirming that the randomized inputs exercised several decoder paths.

The ALU was verified in a similar manner by generating randomized operand values and ALU control signals. A covergroup monitored the selected ALU operation with bins corresponding to operations such as add, subtract, and logical functions. Coverage reports showed hits across several operation bins, demonstrating that different ALU modes were activated during simulation.

These results confirm that the coverage-driven framework is capable of tracking execution behavior and producing measurable coverage metrics during testing.

E. Architectural Definition

The primary deliverable of the review phase from the RTL workstream is the fixed system architecture that subsequent implementation will target. By adopting the Wally RISC-V core, cache integration is inherited rather than built from scratch, allowing development effort to concentrate on the HYDRA engine itself. The memory map in Table I configures the address space around Wally's existing cache window, defining where the scratchpad and MMR regions live. RTL module stubs for hydra_top, hydra_arbiter, hydra_transform, and hydra_mmr have been defined and committed to the repository. A software performance

baseline will soon be defined and will be executed on Wally directly: loop-unrolled bare-metal C transposition at $-O2$ on a warm L1 cache over 8×16 matrices of 32-bit words, providing a measured cycle count as the speedup denominator.

IV. DISCUSSION

A. RTL Review

Several candidates for a third-party RTL core have been identified, including Wally, NEORV32, and Ibex. After careful consideration, Wally was selected and work to identify the integration strategy for our DMA followed.

B. Design for Testability (DFT)

Future work will focus on improving fault coverage by introducing more diverse scan patterns, performing multi-cycle capture experiments, and extending the testing to additional scan bits.

C. DCache Verification

The current verification framework establishes the basic infrastructure for cache verification. Future work will focus on implementing a scoreboard and reference model to automatically check cache hit/miss behavior and memory refill correctness. Randomized test sequences and coverage metrics will also be introduced to improve verification completeness.

D. Core Verification

Work continues by integrating the developed covergroups into full-core simulations. Coverage reports from these runs are used to identify untested instruction or datapath behaviors and to guide the development of a verification test plan for the processor core.

E. Architecture: Open Problems and Next Steps

Two open problems remain before RTL implementation begins. First, AHB-Lite's half-duplex constraint means the ping-pong buffers hide transformation latency but cannot overlap bus-side reads and writes: the throughput gap relative to the Equation 1 ceiling must be quantified by simulation. Second, burst atomicity under arbitration, a known failure mode in multi-master DMA architectures [5], requires HY-DRA's address generation unit to maintain restart state at block boundaries, allowing the CPU to be granted the bus between blocks without restarting a full transfer. Immediate next steps are standalone simulation of `hydra_arbiter`, implementation of Mode C to validate the ping-pong interface, and execution of the software baseline on Wally to establish measured cycle counts.

V. ACKNOWLEDGMENTS

We would like to express our gratitude towards Professor Welsh and Professor Simar for their support of our project.

VI. CREDIT AUTHOR STATEMENT

- **Nafil Atiq:** RTL review and core evaluation.
- **Ruohan Dong:** UVM verification for RISC-V data cache.
- **Matthew Nutt:** RTL review and DMA architecture design.
- **Giovanni Sirtori:** Literature review and DMA architecture design.
- **Ithzel Toscano:** UVM verification of RISC-V core.
- **Yulia Zhou:** Design for Testability (DFT).

REFERENCES

- [1] C. E. Tripp, J. Perr-Sauer, J. Gafur, A. Nag, A. Purkayastha, S. Zisman, and E. A. Bensen, "Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations," *arXiv:2403.08151*, 2024.
- [2] S. Ma, Y. Lei, L. Huang, and Z. Wang, "MT-DMA: A DMA controller supporting efficient matrix transposition for digital signal processing," *IEEE Access*, vol. 7, pp. 5808–5818, 2019.
- [3] D. Harris, S. Stine, and M. Thompson "Wally: Configurable RISC-V processor," *Harvey Mudd College*, 2022. [Online]. Available: <https://github.com/openhwgroup/cvw>
- [4] T. Benz, M. Rogenmoser, P. Scheffler, S. Riedel, A. Ottaviano, A. Kurth, T. Hoefler, and L. Benini, "A high-performance, energy-efficient modular DMA engine architecture," *IEEE Transactions on Computers*, 2023, *arXiv:2305.05240*.
- [5] J. Ward and J. Bowen, "TMS320C64x EDMA architecture," Texas Instruments Application Report SPRA994, Mar. 2004.